FemboyEngine2D

Application Guide

ECS in practice, production optimisation and the distribution SDK

0.3.0-alpha.1 · 2026-11-06

1. What this guide is for

FemboyEngine2D is a data-oriented 2D engine. This guide is the practical companion to the architecture specification: it starts from a playable 2D shooter, shows how entities are composed from independent data, then follows the same project through texture cooking, GPU batching, storefront integration, hybrid SDF physics, the weather pipeline and the encrypted distribution archive.

The running project is deliberately small. A player, three enemy archetypes, a boss with phases, bullets, pickups and a card board are enough to demonstrate every architectural rule. Every code snippet compiles against the headers shipped in Source/Engine; paths and type names are real.

- Audience: gameplay programmers, technical designers and tools engineers.
- Assumed knowledge: C++20 basics, an ECS at the level of EnTT, and one graphics API.
- Not covered: Vulkan synchronization details, console certification and legal storefront setup.

Note. The engine is validated by the full CTest suite. The suites cover the SDK profile and crypto, the storefront bridge, SDF physics, the render/weather stack and the data-driven prefab/DPI/theme path. Run ctest --test-dir build --output-on-failure before trusting a local change.

2. Setup and the reproducible toolchain

The setup scripts are split by OS and software group. The C++ scripts install the pinned toolchain and materialise the exact sources under third_party/. CMake prefers those local checkouts, so the first configure after setup does not need network access. Versions are fixed by the scripts and written to third_party/dependencies.lock.

```
# Windows 10/11 (run from a Visual Studio developer prompt)
SetupScripts\Windows\Setup.bat
SetupScripts\Windows\SetupCpp.bat
SetupScripts\Windows\SetupArchiver.bat
# optional vendor SDKs
SetupScripts\Windows\SetupUpscalers.bat

# Ubuntu / Debian
./SetupScripts/Linux/SetupCpp.sh
./SetupScripts/Linux/SetupArchiver.sh
# optional vendor SDKs
./SetupScripts/Linux/SetupUpscalers.sh

# macOS (double-click in Finder also works)
./SetupScripts/macOS/SetupCpp.command
./SetupScripts/macOS/SetupArchiver.command
# optional vendor SDKs
./SetupScripts/macOS/SetupUpscalers.command
```

The all-in-one entry points are SetupScripts/Windows/Setup.bat, SetupScripts/Linux/Setup.sh and SetupScripts/macOS/Setup.command.

Pinned versions: SDL3 release-3.4.18, Vulkan-Headers v1.4.366, glslang 15.3.0, miniaudio 0.11.23, Box2D v3.2.0 and LZMA SDK 25.01. Setup installs system packages when possible; use --skip-system (or -SkipSystem on Windows) on a CI image that already has CMake, Ninja, Clang/GCC and the Vulkan loader.

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DCMAKE_C_COMPILER=clang-cl \
  -DCMAKE_CXX_COMPILER=clang-cl \
  -DFBY_BUILD_TESTS=ON
cmake --build build
ctest --test-dir build --output-on-failure
```

The compiler is part of the contract: C++26 is required, Clang 19+ is the reference toolchain and GCC 16+ is supported. The build defines FBY_HAS_MDSPAN, FBY_HAS_FORMAT and FBY_HAS_JTHREAD by probing the compiler, not by version number. LZMA SDK presence is also probed; without it the .femarc writer uses the deterministic FbyLZ codec and reports it in the cook log.

> **Note.** Setup never modifies a source file. It is safe to delete third_party/<name>-src and re-run with --force; the lock file is regenerated and the CMake cache does not embed a machine-specific path.

2.1 Editor and cooked game on Windows, Linux and macOS

The same presets build the editor, the sandbox, the asset tools and the game packer. Windows uses clang-cl with Vulkan; Linux uses clang++ with Vulkan; macOS uses clang++ with Metal. Every preset exports compile_commands.json and keeps the tests enabled.

```
# Windows 10/11, from an LLVM + Visual Studio developer shell
cmake --preset windows-release
cmake --build --preset windows-release
ctest --preset windows-release

# Linux, from a shell with Clang, Ninja and Vulkan headers
cmake --preset linux-release
cmake --build --preset linux-release
ctest --preset linux-release

# macOS, from a shell with Xcode command-line tools and Homebrew
cmake --preset macos-release
cmake --build --preset macos-release
ctest --preset macos-release
```

The editor is build-<platform>/bin/fby_editor (Windows adds .exe). Build & Pack Game uses the same editor on all three systems: it cooks assets through fby_fempak, writes the AOT module when the project has a LogicGraph, and stages archives under the project stores/. The POSIX release path uses posix_spawn and reports the child exit code; the Windows path uses CreateProcess.

> **Note.** Windows is the locally validated gate. Linux and macOS have presets, platform code and CI jobs; treat their first hosted runs as the acceptance gate until the matrix is green. The RHI is Metal on macOS and Vulkan on Windows/Linux.

3. How to think in ECS

An ECS is not a different syntax for a class hierarchy. It is a change of the central question. In OOP the question is 'what is this object?'. In ECS the questions are 'what data does this entity own?' and 'which system transforms that data this tick?'. The answer to the first question is a set of component types; the answer to the second is a system that iterates every entity with that set.

3.1 Rules that keep the model fast

- A component is trivially copyable and standard layout: no string, no vector, no virtual method. Transform2D, HealthComponent and WeaponComponent are values, not objects.
- Behaviour lives in a system. A component never has an Update() method; a system never owns gameplay state that belongs to an entity.
- Identity is data. A player is not a Player class; it is an entity with InputComponent, HealthComponent and WeaponComponent. Anyone with those three can be controlled.
- Variation is data. An enemy archetype is a row in a definition table, and a boss phase is a component value, not a derived class.
- The hot path allocates nothing. Archetype capacity is reserved at load; per-frame work uses views, spans and the frame arena.

3.2 The component vocabulary

Start with a small vocabulary and resist adding a component until two systems genuinely need to share the data. The shooter uses the following independent components.

```
// Game/Components.hpp - trivially copyable, no behaviour.
struct HealthComponent
{
    float current = 100.0f;
    float maximum = 100.0f;
    float invulnerableSeconds = 0.0f;
    std::uint32_t team = 0;
};

struct WeaponComponent
{
    std::uint64_t projectileArchetype = 0;
    float cooldownSeconds = 0.0f;
    float fireInterval = 0.12f;
    Vector2D muzzleOffset{18.0f, 0.0f};
    std::uint32_t pellets = 1;
};

struct InputComponent
{
    Vector2D move{};
    Vector2D aim{1.0f, 0.0f};
    std::uint32_t buttons = 0; // bit 0 fire, bit 1 dash, bit 2 interact
};
```

```
struct AIBehaviorComponent
{
    std::uint32_t behavior = 0; // 0 chase, 1 keep distance, 2 patrol, 3 boss
    Entity target = entt::null;
    float thinkSeconds = 0.0f;
    float preferredRange = 220.0f;
};

struct ProjectileComponent
{
    Entity owner = entt::null;
    Vector2D velocity{};
    float damage = 10.0f;
    float ttl = 4.0f;
    std::uint32_t pierce = 0;
};
```

Notice what is absent: no Enemy base class, no PlayerController, no BossHealthBar. Those are behaviours and presentation assembled from the same data. The compiler enforces the rule with static_assert(std::is_trivially_copyable_v<T>) in the component header.

4. Building a 2D shooter by composition

The player is four components and a transform. Spawning it is a function, not a constructor. The function is also the prefab loader: a scene file produces the same component set through the same code path.

```
Entity SpawnPlayer(ecs::Registry& registry, Vector2D spawn)
{
    const Entity entity = registry.create();
    registry.emplace<Transform2D>(entity, Transform2D{spawn, {1.0f, 1.0f}, 0.0f});
    registry.emplace<Velocity>(entity);
    registry.emplace<HealthComponent>(entity, HealthComponent{100.0f, 100.0f,
0.0f, 0});
    registry.emplace<InputComponent>(entity, InputComponent{});
    registry.emplace<WeaponComponent>(entity, WeaponComponent{0x51A7u, 0.0f,
0.12f, {18.0f, 0.0f}, 1});
    registry.emplace<GameplayTagsComponent>(entity, GameplayTagsComponent{});
    return entity;
}

// Enemy archetypes are data. A boss is the same composition with different
values.
struct EnemyArchetype
{
    std::uint64_t id;
    float health;
    float speed;
    std::uint32_t behavior;
    float preferredRange;
    std::uint32_t weaponArchetype;
    Color32 tint;
};

Entity SpawnEnemy(ecs::Registry& registry, const EnemyArchetype& archetype,
Vector2D spawn)
{
    const Entity entity = registry.create();
    registry.emplace<Transform2D>(entity, Transform2D{spawn, {1.0f, 1.0f}, 0.0f});
    registry.emplace<Velocity>(entity);
    registry.emplace<HealthComponent>(entity, HealthComponent{archetype.health,
archetype.health, 0.0f, 1});
    AIBehaviorComponent behaviorComponent{};
    behaviorComponent.behavior = archetype.behavior;
    behaviorComponent.preferredRange = archetype.preferredRange;
    registry.emplace<AIBehaviorComponent>(entity, behaviorComponent);
    Sprite2D sprite{};
    sprite.Size = {32.0f, 32.0f};
    sprite.color = archetype.tint;
    registry.emplace<Sprite2D>(entity, sprite);
    return entity;
}
```

Mobs and bosses share one spawn path. A 'Boss' is an archetype with more health, a phase table and a BossPhaseComponent; it is not a separate C++ type. Adding a new enemy is a data row and, when its behaviour is genuinely new, one branch in AISystem.

4.1 Systems, in order

- InputSystem (PreUpdate): read devices into InputComponent. It never moves anything.
- AISystem (FixedUpdate): write desired velocity and aim into the same InputComponent that a player would write. This is the key trick: one movement system handles players and enemies.
- MovementSystem (FixedUpdate): integrate Velocity into Transform2D; clamp to level bounds.
- WeaponSystem (FixedUpdate): cooldown, spawn projectile intents through the CommandBuffer.
- ProjectileSystem (FixedUpdate): integrate projectiles, TTL, broad-phase candidate query.
- DamageSystem (FixedUpdate, after ProjectileSystem): apply hits, invulnerability and death flags.
- CameraStackSystem (PostUpdate): follow, shake, tilt and post-FX policy.
- Extract/Render (Render): read-only pass that emits RenderItems and MeshRenderItems.

```
void AISystem(ecs::SystemContext& context)
{
    auto view = context.registry.view<AIBehaviorComponent, InputComponent,
Transform2D, HealthComponent>();
    for (const Entity entity : view)
    {
        const AIBehaviorComponent& behaviorComponent =
view.get<AIBehaviorComponent>(entity);
        const Transform2D& transform = view.get<Transform2D>(entity);
        const HealthComponent& health = view.get<HealthComponent>(entity);
        if (health.current <= 0.0f)
        {
            continue;
        }
        InputComponent& input = view.get<InputComponent>(entity);
        if (behaviorComponent.target == entt::null || !
context.registry.valid(behaviorComponent.target))
        {
            continue;
        }
        const Transform2D& target =
context.registry.get<Transform2D>(behaviorComponent.target);
        const Vector2D delta = target.Position - transform.Position;
        const float distance = std::sqrt(delta.LengthSquared());
        if (distance > behaviorComponent.preferredRange)
        {
            input.move = delta / std::max(distance, 0.001f);
        }
        else if (distance < behaviorComponent.preferredRange * 0.8f)
        {
            input.move = delta / std::max(distance, 0.001f) * -1.0f;
        }
        else
        {
            input.move = Vector2D{};
```

```
        }
        input.aim = delta / std::max(distance, 0.001f);
        input.buttons = distance < behaviorComponent.preferredRange * 1.2f ? 1u :
0u;
    }
}
```

The system is deliberately boring. There is no state machine object and no per-enemy allocation; the archetype row supplies the numbers, and the same nine lines cover a chaser, a turret and a ranged mob. A boss with phases adds a second small system that changes preferredRange and behavior when its health crosses a threshold.

4.2 Bosses without inheritance

```
struct BossPhaseComponent
{
    std::uint32_t currentPhase = 0;
    std::uint32_t phaseCount = 0;
    float healthThresholds[4] = {0.66f, 0.33f, 0.12f, 0.0f};
    std::uint32_t behaviors[4] = {0u, 1u, 2u, 3u};
    float preferredRanges[4] = {220.0f, 160.0f, 90.0f, 40.0f};
};

void BossPhaseSystem(ecs::SystemContext& context)
{
    auto view = context.registry.view<BossPhaseComponent, HealthComponent,
AIBehaviorComponent>();
    for (const Entity entity : view)
    {
        BossPhaseComponent& phase = view.get<BossPhaseComponent>(entity);
        const HealthComponent& health = view.get<HealthComponent>(entity);
        AIBehaviorComponent& behaviorComponent =
view.get<AIBehaviorComponent>(entity);
        const float ratio = health.current / std::max(health.maximum, 0.001f);
        while (phase.currentPhase + 1 < phase.phaseCount &&
               ratio <= phase.healthThresholds[phase.currentPhase])
        {
            ++phase.currentPhase;
            behaviorComponent.behavior = phase.behaviors[phase.currentPhase];
            behaviorComponent.preferredRange =
phase.preferredRanges[phase.currentPhase];
        }
    }
}
```

This is the entire boss hierarchy. A 'fire boss' and a 'charge boss' differ by component values and, at most, by a spawn table. Behaviour that cannot be expressed as data belongs in a new small system; it almost never justifies a class with a vtable.

4.3 Spawning without a frame hitch

Structural changes during iteration are the classic ECS bug. The shooter routes every spawn and despawn through the CommandBuffer and applies it at a phase boundary. The binary CommandLog records the same commands for rollback, so a replay sees the identical entity ids.

```
if (context.commands != nullptr)
{
    context.commands->Spawn(projectileEntity);
    context.commands->AddComponent(projectileEntity, ProjectileComponent{owner,
direction * 900.0f, 12.0f, 4.0f, 0});
}
// At the end of FixedUpdate the host calls commands->Apply(registry).
```

5. Scheduling, determinism and tests

SystemGraph is the production scheduler. Dependencies are explicit; Add() stores a phase, a function pointer and after/before bitmasks. The graph is built once, not every frame. A cycle or a cross-phase dependency fails Build() and is reported instead of silently reordering gameplay.

```
ecs::SystemGraph graph;
graph.Add(ecs::Phase::PreUpdate, "Input", &InputSystem);
graph.Add(ecs::Phase::FixedUpdate, "AI", &AISystem);
graph.Add(ecs::Phase::FixedUpdate, "Movement", &MovementSystem);
graph.Add(ecs::Phase::FixedUpdate, "Weapon", &WeaponSystem);
graph.Add(ecs::Phase::FixedUpdate, "Projectile", &ProjectileSystem);
graph.Add(ecs::Phase::FixedUpdate, "Damage", &DamageSystem);
graph.Add(ecs::Phase::PostUpdate, "Camera", &CameraTick);
if (!graph.Build())
{
    LogError("schedule", "system graph has a cycle or a cross-phase dependency");
}
```

Determinism is a property of the data, not of a hope. FixedUpdate receives a constant dt, the RNG is Pcg32 seeded per run, and structural commands are logged. Snapshot v4 captures the ECS plus the world extension (joints, cloth, fluid) and restores it with a b2 body remap. The replay test resimulates the same input log on a fresh registry and compares the checksum.

- Headless first: a gameplay system takes SystemContext and must not require a window or a GPU.
- Test the data, not the pixels: assert component values and command-log bytes.
- One test executable per concern; no test framework and no dynamic registration.
- Run the whole suite with ctest; a single failure prints file and line and returns non-zero.

5.1 Deterministic input and replication while building the game

Build input and replication together, not after the gameplay is finished. The action mapper produces the same fixed-tick data on every peer; the network layer carries an input frame, never raw SDL events. This is the contract that makes prediction and rollback possible.

```
InputSystem input;
const std::uint16_t Move = input.RegisterAction("Move", InputActionType::Axis2D);
const std::uint16_t Jump = input.RegisterAction("Jump", InputActionType::Bool);
const std::uint16_t Gameplay = input.RegisterContext("Gameplay", 0, false);

InputUserComponent user;
user.UserId = 0;
user.GamepadSlot = 0;
registry.emplace<InputUserComponent>(player, user);
input.BindUser(player, 0, 0);
input.PushContext(0, Gameplay);

// Once per fixed tick, before gameplay reads actions.
input.Update(window.Input(), tick, timestampNs, fixedDt);
const InputActionState& move = *input.Action(0, Move);
if (input.Action(0, Jump)->Triggered) { JumpPlayer(player); }
```

The local device layer is allowed to differ: keyboard, mouse and gamepad take different paths into InputState. The simulation sees only action values and events. For a networked game, pack the values you need into NetInputFrame (or your own schema) and submit the frame for the owning player.

```
NetRollbackSession::Config net;
net.playerCount = 2;
net.localPlayer = 0;
net.rollbackTicks = 64;
NetRollbackSession session;
session.Initialize(registry, Ecs::SnapshotSchema::BuiltIns(), net);
session.SetCommandLog(&commands);
session.RegisterEntity(player, NetEntityId{1});

NetInputFrame frame;
frame.tick = tick;
frame.player = 0;
frame.moveX = static_cast<std::int16_t>(move.X * 32767.0f);
frame.buttons = input.Action(0, Jump)->Triggered ? 1u : 0u;
session.SubmitRemoteInput(0, frame);
```

- Authoritative simulation: the server owns the tick, the RNG seed and the CommandLog. Clients predict with the same tick size and the same gameplay systems.
- Register gameplay components in SnapshotSchema before the first capture; a component outside the schema silently does not replicate.
- Use stable NetEntityId values for replicated entities. Never derive one from an EnTT id, a spawn order or an allocator address.
- Replicate input frames and structural commands, not raw device state or render-only values.
- On a correction, restore the snapshot before the first bad tick, replay the corrected input and compare the final checksum.
- Test PIE with two players, artificial latency, packet loss and a forced rollback before adding content.

A useful project order is: one player + InputSystem; two local users; snapshot round trip; server-authoritative movement; then interaction, weapons and AI. Add replication fields with the feature that owns them, not in a single late 'networking pass'. If a system reads wall-clock time, frame delta or a render handle, it is not ready for rollback.

6. Memory and draw-call compression

The second half of an ECS project is not gameplay; it is what reaches the GPU. Two tools do most of the work: cooked block-compressed textures and indirect mesh batching. Both are configured per project and both are verifiable from FrameStats.

6.1 Texture cooking: BC7 on desktop, ASTC on mobile

- BC7 keeps a real alpha channel and is the desktop default. BC7 sRGB is selected for colour sprites; a normal map or a mask uses the linear (UNORM) variant.
- ASTC 4x4 is the quality mobile target; 6x6 is the size target. Pick it in Project Settings -> Cooker -> Texture format.
- The cooker generates the full mip chain once, at import. The runtime uploads the blocks verbatim; there is no CPU decode and no runtime re-compression.
- A texture array is the batching unit. Keep sprites of one material in one layer set; a layer switch is a new descriptor, not a new draw call.
- Never ship a JPEG: the cooker rejects it by extension and by magic. A lossy source makes a shipped texture differ from what the artist approved.

```
# project.femproj - the same values the panel writes
[render]
texture_format = "bc7-srgb"
max_texture_size = 2048

[build]
optimization = "O2"
lto = true
strip_debug_info = true
unity_build = true
native_arch = false
```

6.2 Multi-draw indirect batching

The sprite path is already one instanced call per batch. The mesh path is where indirect drawing matters: static geometry is uploaded once into a device-local arena and referenced by MeshHandle; deformable geometry is streamed into the transient ring; every draw is a compact MeshDrawData record, and the backend records them through one drawIndexedIndirect call per material layer. The CPU never binds a vertex buffer per mesh.

- One material per layer: a new layer is a new indirect batch but not a pipeline switch.
- Static props use CreateMesh and a MeshHandle; a per-frame upload is wasted PCIe bandwidth.
- Skinned meshes are skinned by one compute dispatch into the skinned vertex buffer, then drawn through the same indirect path.
- Watch FrameStats: items, batches, drawCalls, meshIndirectCalls. A batch count that grows with entity count means the authoring data is fragmented, not that the GPU is slow.
- The render graph orders passes; it does not reorder draws inside a pass. Keep the draw list sorted by pipeline/layer once per frame.

Note. The procedural Pink UI path removes texture memory entirely: circles, rounded boxes and cards are evaluated from a 64-byte instance record in pink_sdf.vert/frag. A card board of 200 slots is two instanced draws (cards and shadows), not 200 texture switches.

7. Storefront integration: Steam and Epic

PlatformService is a bridge, not a wrapper around a wrapper. The engine owns a single C-ABI function table loaded from a provider shared library (fby_platform_steam or fby_platform_egs). Gameplay enqueues commands; Pump() is the only place a vendor SDK is called, and it runs after the frame has been recorded.

```
PlatformService platform;
std::string error;
if (!platform.Load(FbyPlatformKind_Steam, "fby_platform_steam.dll", &error))
{
    LogWarn("platform", "Steam provider unavailable: {}", error);
}
FbyPlatformInitDesc desc{};
desc.kind = FbyPlatformKind_Steam;
desc.applicationId = "480";
if (platform.Initialize(desc, &error))
{
    PlatformServiceCallbacks callbacks{};
    callbacks.onAchievement = &OnAchievement;
    platform.SetGameCallbacks(callbacks);
}
// Once per PreUpdate, after gameplay has enqueued this frame's intents:
platform.Pump();
```

7.1 Achievements as deterministic commands

SteamAchievementSystem reads AchievementRequest components, sorts them by achievement id (so ECS iteration order cannot change the log), pushes one command per request and appends a PlatformCommandLog record. The log is binary, checksummed and replayable; it answers 'why did this unlock?' without trusting a vendor dashboard. EgsStoreSystem does the same for cloud saves, with a fixed staging arena and a bounded write queue.

```
SteamAchievementSystem achievements;
achievements.Initialize(platform, commandLog);
EgsStoreSystem store;
store.Initialize(platform, commandLog);

ecs::SystemGraph graph;
SteamAchievementSystem::Register(graph, achievements);
EgsStoreSystem::Register(graph, store);
graph.Build();

// Gameplay only writes data; the systems do the rest.
AchievementRequest request{};
request.achievementId = 0xA17E;
request.Flags = AchievementPending;
std::strcpy(request.apiName, "ACH_FIRST_BLOOD");
registry.emplace<AchievementRequest>(player, request);

// Cloud save: stage once, then mark the component dirty.
store.StageWrite(0x51u, "slot1.sav", std::span<const std::byte>(saveBytes));
```

```
CloudSaveComponent save{};
save.saveId = 0x51u;
save.Flags = CloudSaveDirty;
std::strcpy(save.fileName, "slot1.sav");
registry.emplace<CloudSaveComponent>(player, save);
```

A frame never blocks. If the provider is absent, QueueAchievement returns false, the request is marked failed and the game keeps running. The C ABI is versioned; a provider built for another ABI is rejected before a call is made. Steam is resolved through the flat API at runtime, so the game does not link steam_api; EGS uses EOS_Platform_Create with an options blob supplied by the game and can be built with the official headers when FBY_EOS_SDK_INCLUDE_DIR is set.

8. Hybrid SDF physics, rays and post effects

Box2D resolves body-vs-body contacts. Terrain is not a body: it is a distance field the renderer already samples. SdfCollisionSystem closes the gap once per fixed step. It reads a non-owning SdfField over std::mdspan, bilinearly samples it at rotated collider corners and the centre, takes the gradient as the normal, applies a mass-scaled push impulse and a positional correction, then writes the corrected state back to RigidBody2D.

```
// A height field is filled by the level loader once.
std::vector<float> samples(width * height);
for (std::uint32_t y = 0; y < height; ++y)
{
    for (std::uint32_t x = 0; x < width; ++x)
    {
        samples[y * width + x] = DistanceToTerrain(x, y); // negative inside
    }
}
SdfField field{};
field.samples = samples.data();
field.width = width;
field.height = height;
field.origin = levelOrigin;
field.cellSize = {2.0f, 2.0f};

SdfCollisionSystem sdf;
sdf.SetField(field);
sdf.Register(graph); // FixedUpdate, after the Box2D step
// Each Step produces up to 1024 SdfContact records for audio/VFX/decal reuse.
for (const SdfContact& contact : sdf.Contacts())
{
    SpawnImpactVfx(contact.point, contact.normal, contact.impulse);
}
```

8.1 SDF shadows and camera-owned post FX

SdfShadowPass raymarches the same field in a compute shader and produces a soft light mask; the composite multiplies the scene by it. PostFxPass adds a bloom prefilter, a separable blur and one composite for CRT curvature, scanlines, vignette, chromatic aberration and grain. The settings live in PostFxSettings on the top CameraStack entry, so a terminal close-up and a gameplay camera can have different looks without rebuilding the graph.

```
CameraStackSystem cameras;
CameraStackDesc camera{};
cameras.Push(registry, camera);

PostFxSettings terminal{};
terminal.Flags = PostFxCrt | PostFxVignette | PostFxBloom |
PostFxChromaticAberration;
terminal.crtCurvature = 0.12f;
terminal.scanlineStrength = 0.18f;
cameras.SetPostFx(terminal);
```

```
SdfShadowPass shadows;
SdfShadowDesc shadowDesc{};
shadowDesc.fieldWidth = width;
shadowDesc.fieldHeight = height;
shadowDesc.fieldCellSize = {2.0f, 2.0f};
shadows.Initialize(shadowDesc);
shadows.Build(graph);

PostFxPass postFx;
postFx.Build(graph, cameras);
```

For UI, the SDF path removes texture memory entirely. pink_sdf.frag evaluates circles, rounded boxes, capsules and rings analytically; anti-aliasing comes from fwidth. SlotComponent and CardBoardComponent lay out a card board without a scene, and BuildCardBoardPrimitives packs each slot into two instances (card and shadow).

8.2 Moving platforms, transport and wheels

A carrier is a transform frame, not a second rigid-body world. TransportGroupSystem is for elevators, trains and platforms: it computes carrier deltas before the physics step and refreshes local poses after it. A wheel entity is a normal dynamic body with a collider; WheelSystem creates the Box2D wheel joint lazily and applies the per-wheel spring, drive and brake settings before the step.

```
TransportGroupSystem transport;
transport.CreateGroup(registry, elevator, 1);
transport.Attach(registry, player, 1, kCarrierModeSupport);

TransportSurface surface;
surface.Friction = 0.8f;
registry.emplace<TransportSurface>(elevator, surface);

Vehicle2D vehicle;
vehicle.MaxMotorTorque = 600.0f;
registry.emplace<Vehicle2D>(chassis, vehicle);

Wheel2D wheel;
wheel.Chassis = chassis;
wheel.LocalAnchor = {-20.0f, 4.0f};
wheel.Radius = 8.0f;
wheel.DriveWeight = 1.0f;
registry.emplace<Wheel2D>(wheelEntity, wheel);

WheelSystem wheels;
wheels.Initialize(physics);
wheels.Update(registry, physics, fixedDt);
physics.Step(registry, fixedDt);
```

9. The weather pipeline

Weather is one component: WeatherContextComponent stores atmosphere density, viscosity, gravity scale, cloud coverage, fog density, rain/snow/fire intensity and the global wind. WeatherSystem advances it in PreUpdate, derives a wind vector from fBm noise, writes the resolved vector back and pushes it into ClothSystem. Every downstream consumer consumes that one resolved vector, so cloth, foliage and rain never disagree.

```
WeatherSystem weather;
weather.Initialize(WeatherDesc{0x51ED270Bu, Vector2D{1.0f, 0.0f}, 14.0f, 1.0f});
WeatherContextComponent context{};
context.atmosphereDensity = 0.82f;   // thinner air on an alien map
context.gravityScale = 0.65f;
context.windSpeed = 22.0f;
context.turbulence = 0.55f;
context.rainIntensity = 0.9f;
context.cloudCoverage = 0.7f;
context.fogDensity = 0.025f;
registry.emplace<WeatherContextComponent>(weatherEntity, context);

weather.Update(systemContext); // PreUpdate
// WindField() is now the shared block consumed by cloth, plants and GPU weather.
```

9.1 GPU rain with SDF splashes

WeatherPreconciliatedSystem owns no particle array on the CPU. It prepares WeatherGpuConstants, computes a dispatch size from intensity and records four graph passes. weather_rain.comp owns the particle SSBO, integrates wind and gravity, samples the global terrain SDF and appends a splash record with atomicAdd when a drop crosses the surface. A second compute ages splashes, and two graphics passes render lines and ring SDF sprites. Stats.cpuParticleReads stays zero by construction.

```
WeatherPreconciliatedDesc rain{};
rain.particleCapacity = 262144;
rain.splashCapacity = 65536;
rain.areaWidth = 4096.0f;
rain.areaHeight = 2304.0f;

WeatherPreconciliatedSystem gpuRain;
gpuRain.Initialize(rain);
gpuRain.SetSdfField(field);
gpuRain.BuildRenderGraph(graph);

WeatherAtmospherePass atmosphere;
atmosphere.Build(graph, weather); // clouds, cloud shadows, fog, fire, composite

// Per frame, once:
gpuRain.Update(systemContext); // constants + dispatch size, zero particle reads
```

- Clouds: two-octave simplex advected by the resolved wind, shaped by coverage and a Worley edge.
- Cloud shadows: the same field sampled with a drifted Poisson kernel; shadows move with wind.
- Fog: 24-step vertical raymarch through exponential height fog with value-noise detail.

- Fire/smoke: simplex flame plus slow smoke; intensity comes from the weather component.
- All four are compute passes writing transient textures; no heavy source textures are loaded.
- The Metal kernel weather_rain.metal mirrors the Vulkan compute contract for Apple targets.

10. Distribution: .femarc, LZMA and AEAD

.fempak is the mmap-friendly runtime archive: no compression, deterministic bytes, sorted paths. .femarc is the distribution envelope: per-entry LZMA (or the in-tree FbyLZ fallback) plus AES-256-GCM or ChaCha20-Poly1305. The AEAD tag is verified before a single entry offset is trusted. The reader works on caller-provided buffers, so the launcher decrypts into a mapped file without a heap allocation.

```
fby_fempak secure --in build/release/assets.fempak --out archives/game.femarc \
  --entry game/assets.fempak --codec lzma --cipher aes \
  --key-env FBY_ARCHIVE_KEY
```

The key is supplied through an environment variable or a 64-hex argument; the nonce is generated from SecureRng when omitted. Build is idempotent for a fixed key/nonce/entry set: the table is sorted, the string table is stable and the compressed stream is deterministic. Tampering with one byte makes Open() return AuthenticationFailed, not a half-parsed archive.

> **Note.** LZMA SDK is optional. When third_party/lzma-sdk/LzmaLib.h exists, CMake compiles the 7-Zip encoder and decoder; otherwise the build log says 'FbyLZ fallback is active' and the archive remains valid but larger. A shipped build should pin and audit one codec, not switch per run.

11. Performance playbook and CI

- Profile fixed-step time separately from frame time. Physics, cloth and AI belong to the first; post FX and weather to the second.
- Keep fixed-step work allocation-free. Reserve archetype capacity at scene load; use the frame arena for transient spans.
- Watch three counters before changing code: FrameStats.drawCalls, meshIndirectCalls and transient capacity. A rising drawCalls with a flat item count means fragmented batches.
- Use the O2 profile for rollback and CI. Ofast can change float comparison results and breaks replay determinism; keep it for a visual-only pass.
- LTO and unity builds are release-only. They cut ship time but multiply link time and make incremental cooking unpleasant.
- Never let a vendor callback run inside gameplay. Enqueue, return, Pump after the frame.

11.1 Release checklist

Run the whole CTest slate, then the cook and the secure step on a clean machine. Verify that the game starts without Steam/EOS (provider absent is a warning, not a crash), that a tampered .femarc is rejected, and that the SDF physics suite still passes with the exact shipping profile. Record the profile and the dependency lock in the build metadata.

```
ctest --test-dir build --output-on-failure
fby_fempak cook --root assets --out build/release/assets.fempak
fby_fempak secure --in build/release/assets.fempak --out archives/game.femarc \
  --codec lzma --cipher chacha --key-env FBY_ARCHIVE_KEY
# ReleaseCooker then stages archives/game.femarc and the plugin into
# stores/steam, stores/epic and stores/gog with a store.txt manifest.
# Launch the game with FBY_PLATFORM_LIB pointing at fby_platform_steam.dll
# or omit it and verify the NotAvailable path.
```

12. Materials: author, cook and bind to a mesh

A material is a small graph that replaces the fragment shader of a mesh: base colour, opacity and emissive, plus math nodes, atlas samples and the 2.5D helpers. The graph is authored in the editor, cooked at build time and loaded by the game. Nothing in the game compiles GLSL.

The three artefacts

File	Produced by	Loaded by
`<name>.fgraph`	The editor panel (Save .fgraph) or `EncodeMaterialGraph`	`fby_matc`, and the editor panel (Load .fgraph)
`<name>.femmat`	`fby_matc` in the build step	`Render::DecodeCookedMaterial` in the game
`MaterialPipelineCache`	The game, one per device	The render extract via `MeshRenderItem`

The build step runs `fby_matc <graph>.fgraph -o <graph>.femmat` (add `--sprite` for the quad.vert contract). The tool validates the graph, exits non-zero on failure and writes nothing, so a failed cook cannot ship a stale payload.

Binding it in the game

```
// Once, when the device comes up.
Render::MaterialPipelineCache Materials;

// Decode the payload, then borrow a pipeline reference.
Render::CookedMaterialView Tinted;
if (!Render::DecodeCookedMaterial(InBytes, &Tinted))
{
    return false;
}
const Rhi::ShaderPipelineHandle TintedPipeline = Materials.Acquire(gpu, Tinted);

// The id means use this material; the default keeps the engine shader.
MeshRenderItem Item;
Item.materialPipeline = TintedPipeline.IsValid() ? TintedPipeline.id :
kDefaultMaterialPipeline;

// Shutdown: release while the device is still alive.
Materials.Release(TintedPipeline);
Materials.Shutdown(gpu);
```

- Do not cook per frame or per level load: the `.femmat` is a build artefact, like a compressed texture.
- One cache per device; `Acquire` is safe per item and only bumps the count for the same hash.
- Check `SupportsRuntimeShaderPipelines()` once at startup instead of probing for an invalid handle.

- Sprites use a separate cook: `fby_matc <graph> --sprite` builds the quad.vert contract, `DrawOrder::MaterialBucket` sorts the sprite into a material run, and `SetMaterialPipelines` maps that bucket to the cached pipeline id.

Note. Reference: API reference section 8 lists every type and payload field; Complete Reference chapter 38 walks the cook, the cache and the frame; ADR 0002 records why the runtime does not link glslang.

13. Appendix: component and API map

Use this table as a starting checklist. A component that does not fit one row is probably two components, and a behaviour that needs a new row is a new system, not a new class.

Component	Owner	Written by	Consumed by
Transform2D	every visible entity	Movement, Physics	Render, Camera
Velocity	moving entity	AI, Movement	Physics, Movement
HealthComponent	damageable	DamageSystem	Damage, UI, BossPhase
WeaponComponent	armed entity	Loadout	WeaponSystem
InputComponent	player or AI	InputSystem / AISystem	Movement, Weapon
AIBehaviorComponent	AI entity	Spawner, BossPhase	AISystem
ProjectileComponent	bullet	WeaponSystem	Projectile, Damage
AchievementRequest	player	Gameplay trigger	SteamAchievementSystem
CloudSaveComponent	account	Save system	EgsStoreSystem
WeatherContextComponent	level	Designer / weather curve	WeatherSystem, render
SlotComponent	card	Deck logic	CardBoardSystem, PinkSdf

11.1 Where to look in the repository

- Source/Engine/ECS: Components.hpp, Schedule.hpp, SystemGraph.hpp, CommandLog.hpp.
- Source/Engine/Physics: SdfCollision.hpp/.cpp, Cloth.hpp/.cpp.
- Source/Engine/Platform: PlatformService, PlatformSystems; Source/Plugins/PlatformService: C ABI and providers.
- Source/Engine/RHI: RenderGraph, RenderPasses, PostProcess, CameraStack, shaders/.
- Source/Engine/Weather: Weather, WeatherPreconciliated.
- Source/Engine/UI: CardBoard, PinkSdf, PinkUI.
- Source/Engine/Assets: FemArchiver, FemCrypto, FempakWriter.
- Source/Tests: the executable specification; start with SdkCryptoTest, PlatformBridgeTest, SdfPhysicsTest and RenderWeatherTest.
- Documentation/Tools/GenerateApplicationGuide.py: this document's source.

The guide is generated from the same Python file that writes both languages. Add a section to SECTIONS, run the generator, and the Markdown and PDF stay in lockstep.